

4. Especificação (Spec)

4.1 Modelos de Dados (Model)

/app/models/user.py

- ação: criar
- descrição: Define a entidade de Usuário no banco de dados SQLite via SQLAlchemy. Mapeia os atributos básicos e o papel de acesso (Role) que diferencia administradores, editores e leitores.
- pseudocódigo:

```
CLASSE User HERDA DE db.Model:
    ATRIBUTO id: INTEIRO, CHAVE PRIMÁRIA, AUTOINCREMEN TO
    ATRIBUTO nome: TEXTO (TAMANHO MAX 100), NÃO NULO
    ATRIBUTO email: TEXTO (TAMANHO MAX 100), ÚNICO, NÃO NULO
    ATRIBUTO senha_hash: TEXTO, NÃO NULO
    ATRIBUTO papel: TEXTO (VALORES PERMITIDOS: 'admin', 'editor', 'leitor'), NÃO NULO

    MÉTODO verificar_senha(senha_plana):
        RETORNAR comparar_hash(self.senha_hash, senha_plana)
```

/app/models/request.py

- ação: criar
- descrição: Define a entidade de Solicitação, vinculando um Leitor (remetente) a uma Editora (destinatário), armazenando o status e a resposta fornecida.
- pseudocódigo:

```
CLASSE Request HERDA DE db.Model:
    ATRIBUTO id: INTEIRO, CHAVE PRIMÁRIA, AUTOINCREMEN TO
    ATRIBUTO leitor_id: INTEIRO, CHAVE ESTRANGEIRA (User.id), NÃO NULO
    ATRIBUTO editor_id: INTEIRO, CHAVE ESTRANGEIRA (User.id), NÃO NULO
    ATRIBUTO conteudo: TEXTO, NÃO NULO
    ATRIBUTO resposta: TEXTO, NULO
    ATRIBUTO status: TEXTO (VALORES PERMITIDOS: 'pendente', 'respondida'), PADRÃO 'pendente'
    ATRIBUTO data_criacao: DATA_HORA, PADRÃO DATA_HORA_ATUAL
```

4.2 Controladores (Controllers)

/app/controllers/auth.py

- ação: criar
- descrição: Implementa a lógica de autenticação de usuários, verificando se o e-mail e a senha fornecidos correspondem a um usuário existente no banco de dados.

- descrição: Responsável pelo controle de sessão (login/logout) e pelo autocadastro aberto exclusivamente para o perfil de "leitor".
- pseudocódigo:

```

ROTA POST '/auth/register':
    RECEBER payload (nome, email, senha) DO CLIENTE
    SE email EXISTE EM User:
        RETORNAR HTTP 400 "Email já cadastrado"

    senha_criptografada = GERAR_HASH(senha)
    novo_usuario = User(nome, email, senha_criptografada, papel='leitor')

    INSERIR novo_usuario NO db
    SALVAR db
    RETORNAR HTTP 201 "Leitor cadastrado com sucesso"

ROTA POST '/auth/login':
    RECEBER payload (email, senha) DO CLIENTE
    usuario = BUSCAR User POR email

    SE usuario EXISTE E usuario.verificar_senha(senha) VERDADEIRO:
        GERAR token_sessao
        RETORNAR HTTP 200 COM token_sessao E usuario.papel
    SENÃO:
        RETORNAR HTTP 401 "Credenciais inválidas"

```

/app/controllers/admin.py

- ação: criar
- descrição: CRUD centralizado de usuários e geração de relatórios gerenciais. Rota protegida, permitindo apenas acesso de usuários com papel de 'admin'.
- pseudocódigo:

```

MIDDLEWARE verificar_admin:
    SE sessao_atual.usuario.papel DIFERENTE DE 'admin':
        RETORNAR HTTP 403 "Acesso Negado"

ROTA POST '/admin/users' (USA MIDDLEWARE verificar_admin):
    RECEBER payload (nome, email, senha, papel)
    SE papel NÃO ESTÁ EM ('editor', 'leitor', 'admin'):
        RETORNAR HTTP 400 "Papel de usuário inválido"

    novo_usuario = User(nome, email, GERAR_HASH(senha), papel)
    INSERIR novo_usuario NO db
    SALVAR db
    RETORNAR HTTP 201 "Usuário criado com sucesso"

ROTA GET '/admin/reports' (USA MIDDLEWARE verificar_admin):
    total_usuarios = CONTAR User AGRUPADO POR papel
    total_solicitacoes = CONTAR Request AGRUPADO POR status

```

total_solicitacoes = CONTAR Request WHERE FOR status

```
relatorio = {
    "usuarios": total_usuarios,
    "solicitacoes": total_solicitacoes
}
RETORNAR HTTP 200 COM relatorio
```

/app/controllers/reader.py

- ação: criar
- descrição: Gerencia o envio de solicitações feitas pelos leitores e a consulta do status das mesmas. Rota protegida, acesso restrito a 'leitor'.
- pseudocódigo:

```
MIDDLEWARE verificar_leitor:
    SE sessao_atual.usuario.papel DIFERENTE DE 'leitor':
        RETORNAR HTTP 403 "Acesso Negado"

ROTA POST '/reader/requests' (USA MIDDLEWARE verificar_leitor):
    RECEBER payload (editor_id, conteudo)
    VERIFICAR SE editor_id PERTENCE A UM User COM papel='editor'
    SE FALSO:
        RETORNAR HTTP 404 "Editora não encontrada"

    nova_solicitacao = Request(
        leitor_id=sessao_atual.usuario.id,
        editor_id=editor_id,
        conteudo=conteudo,
        status='pendente'
    )
    INSERIR nova_solicitacao NO db
    SALVAR db
    RETORNAR HTTP 201 "Solicitação enviada"

ROTA GET '/reader/requests' (USA MIDDLEWARE verificar_leitor):
    minhas_solicitacoes = BUSCAR Request ONDE leitor_id == sessao_atual.usuario.id
    RETORNAR HTTP 200 COM minhas_solicitacoes
```

/app/controllers/editor.py

- ação: criar
- descrição: Permite à editora visualizar e responder às solicitações pendentes. Rota protegida, acesso restrito a 'editor'.
- pseudocódigo:

```
MIDDLEWARE verificar_editor:
    SE sessao_atual.usuario.papel DIFERENTE DE 'editor':
```

```
RETORNAR HTTP 403 "Acesso Negado"
```

```
ROTA PUT '/editor/requests/{id}/respond' (USA MIDDLEWARE verificar_editor):
```

```
    RECEBER payload (resposta)
```

```
    solicitacao = BUSCAR Request POR id
```

```
    SE solicitacao NÃO EXISTE OU solicitacao.editor_id DIFERENTE DE sessao_atual.u
```

```
        RETORNAR HTTP 404 "Solicitação não encontrada para esta editora"
```

```
    solicitacao.resposta = resposta
```

```
    solicitacao.status = 'respondida'
```

```
    ATUALIZAR solicitacao NO db
```

```
    SALVAR db
```

```
    RETORNAR HTTP 200 "Solicitação respondida com sucesso"
```

4.3 Inicialização e Configuração (Setup)

```
run.py
```

- ação: modificar
- descrição: Ponto de entrada da aplicação. Deve conter uma rotina de *seed* (semeadura) para garantir que o administrador inicial exista no banco de dados na primeira execução.
- pseudocódigo:

```
IMPORTAR app, db DE app
```

```
IMPORTAR User DE app.models.user
```

```
FUNCAO criar_admin_inicial():
```

```
    admin_email = LER_VARIAVEL_AMBIENTE('ADMIN_INITIAL_EMAIL')
```

```
    admin_senha = LER_VARIAVEL_AMBIENTE('ADMIN_INITIAL_PASSWORD')
```

```
    admin_existe = BUSCAR User ONDE email == admin_email
```

```
    SE admin_existe É FALSO:
```

```
        novo_admin = User(
```

```
            nome='Administrador Master',
```

```
            email=admin_email,
```

```
            senha_hash=GERAR_HASH(admin_senha),
```

```
            papel='admin'
```

```
        )
```

```
        INSERIR novo_admin NO db
```

```
        SALVAR db
```

```
        IMPRIMIR "Admin inicial criado com sucesso."
```

```
INICIAR db.create_all()
```

```
EXECUTAR criar_admin_inicial()
```

```
INICIAR app.run()
```